PRACTICAL 4

AIM: Perform data cleaning tasks using Pandas:

- **Identify duplicates and inconsistent data**
- **Standardize categorical values**
- **Handle heterogeneous data types**

Code:

```
import pandas as pd
import kagglehub
import os

path = kagglehub.dataset_download("shivavashishtha/dirty-excel-data")
file_name = os.listdir(path)[0]
file_path = os.path.join(path, file_name)

df = pd.read_excel(file_path)

df = df.drop_duplicates()
df = df.map(lambda x: x.strip() if isinstance(x, str) else x)

if 'Gender' in df.columns:
    df['Gender'] = df['Gender'].str.capitalize().replace({'M': 'Male', 'F': 'Female'})

for col in df.columns:
    if df[col].dtype == 'object':
        numeric_col = pd.to_numeric(df[col].astype(str).str.replace(r'[$,]', '', regex=True),
errors='coerce')
        if numeric_col.notna().sum() > (len(df) * 0.4):
            df[col] = numeric_col

print(f"Success! Loaded: {file_name}")
print(f"Cleaned Rows: {df.shape[0]} | Cleaned Columns: {df.shape[1]}")
print("\nFirst 5 Records:")
print(df.head())
```

OUTPUT:

```
Using Colab cache for faster access to the 'dirty-excel-data' dataset.
Success! Loaded: Cola.xlsx
Cleaned Rows: 97 | Cleaned Columns: 12

First 5 Records:
   Data provided by SimFin              Unnamed: 1  Unnamed: 2  Unnamed: 3
0                      NaN                     NaN         NaN         NaN
1  Profit & Loss statement                     NaN         NaN         NaN
2                      NaN          in million USD         NaN         NaN
3                      NaN  NET OPERATING REVENUES     30990.0     35119.0
4                      NaN      Cost of goods sold     11088.0     12693.0

   Unnamed: 4  Unnamed: 5  Unnamed: 6  Unnamed: 7  Unnamed: 8  Unnamed: 9
0         NaN         NaN         NaN         NaN         NaN         NaN
1         NaN         NaN         NaN         NaN         NaN         NaN
2         NaN         NaN         NaN         NaN         NaN         NaN
3     46542.0     48017.0     46854.0     45998.0     44294.0     41863.0
4     18215.0     19053.0     18421.0     17889.0     17482.0     16465.0

   Unnamed: 10  Unnamed: 11
0          NaN          NaN
1          NaN          NaN
2          NaN          NaN
3      35410.0      31856.0
4      13255.0      11770.0
```

PRACTICAL 5

AIM:To create a numerical dataset and perform outlier detection using statistical methods such as Interquartile Range (IQR) and Z-Score, and to analyze the impact of outliers on data distribution by comparing robust and non-robust statistical techniques using Python.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)
clean_data = np.random.normal(loc=50, scale=5, size=100)

outliers = np.array([10, 15, 95, 110, 120])
data_with_outliers = np.concatenate([clean_data, outliers])

df = pd.DataFrame({'Values': data_with_outliers})

print("Dataset created with 100 normal points and 5 outliers.\n")

def compare_statistics(data, label):
    mean_val = np.mean(data)
    std_val = np.std(data)
    median_val = np.median(data)
    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)
    iqr_val = q3 - q1

    print(f"Statistics for {label}")
    print(f"Non-Robust -> Mean: {mean_val:.2f}, Std Dev: {std_val:.2f}")
    print(f"Robust     -> Median: {median_val:.2f}, IQR: {iqr_val:.2f}\n")

compare_statistics(clean_data, "Clean Data (No Outliers)")
compare_statistics(data_with_outliers, "Data WITH Outliers")

df['Z_Score'] = np.abs(stats.zscore(df['Values']))

z_threshold = 3
```

```
z_outliers = df[df['Z_Score'] > z_threshold]

print(f"Score Outlier Detection (Threshold = {z_threshold})")
print(f"Number of outliers detected: {len(z_outliers)}")
print(f"Outlier Values: {z_outliers['Values'].values}\n")

Q1 = df['Values'].quantile(0.25)
Q3 = df['Values'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

iqr_outliers = df[(df['Values'] < lower_bound) | (df['Values'] > upper_bound)]

print("IQR Outlier Detection")
print(f"Lower Bound: {lower_bound:.2f}, Upper Bound: {upper_bound:.2f}")
print(f"Number of outliers detected: {len(iqr_outliers)}")
print(f"Outlier Values: {iqr_outliers['Values'].values}")

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

axes[0].hist(df['Values'], bins=20, color='skyblue', edgecolor='black')
axes[0].set_title('Data Distribution (Histogram)')
axes[0].set_xlabel('Values')
axes[0].set_ylabel('Frequency')
axes[0].axvline(np.mean(df['Values']), color='red', linestyle='dashed', linewidth=2,
label='Mean (Non-Robust)')
axes[0].axvline(np.median(df['Values']), color='green', linestyle='dashed', linewidth=2,
label='Median (Robust)')
axes[0].legend()
axes[1].boxplot(df['Values'], vert=False, patch_artist=True,
boxprops=dict(facecolor='lightblue'))
axes[1].set_title('Outlier Detection (Boxplot / IQR Method)')
axes[1].set_xlabel('Values')

plt.tight_layout()
plt.show()
```

OUTPUT:

```
Dataset created with 100 normal points and 5 outliers.

Statistics for Clean Data (No Outliers)
Non-Robust -> Mean: 49.48, Std Dev: 4.52
Robust     -> Median: 49.37, IQR: 5.03

Statistics for Data WITH Outliers
Non-Robust -> Mean: 50.46, Std Dev: 12.11
Robust     -> Median: 49.42, IQR: 5.57

Score Outlier Detection (Threshold = 3)
Number of outliers detected: 4
Outlier Values: [ 10.  95. 110. 120.]

IQR Outlier Detection
Lower Bound: 38.63, Upper Bound: 60.93
Number of outliers detected: 6
Outlier Values: [ 36.90127448  10.          15.          95.         110.
 120.        ]
```